



Proposed Syllabus:

Generative AI Foundations for Engineers

Duration: 8 hours (includes lunch and breaks)

Target Audience: Engineers with minimal prior experience in programming Generative AI (GenAI) tools

Goal: By the end of this workshop, participants will have a basic understanding of how to program GenAI tools and be able to create a simple proof of concept (PoC).

Format: This workshop is designed to be very interactive with minimal lectures, with each limited to 10-15 minutes. Prior to the workshop, learners will divide themselves into teams of 2-5 people, which will be used for collaborative work and scoring of team results. This workshop will focus on three (tentative) topic areas: code generation, analysis of structured data, or deriving intelligence from unstructured data. The workshop itself is divided into modules that will create a guided, hands-on experience. Each module will teach a component that the learners can use to create a coherent, final product or several smaller products. The modules are designed to involve a minimum amount of lecturing, maximizing time spent coding. Teams will be judged and their results scored in order to select one or several winning teams and award prizes. Company-relevant data shall be used insomuch as possible to tie the learnings into ideas that can provide the company value in the future.

Module 1: Generative AI Foundations & LangChain Basics

Objective: Provide a solid foundation in generative AI and LangChain so students can start building AI-powered tools exclusively using AWS-hosted models. This module ensures they understand the fundamental concepts they will apply in subsequent modules.

Lecture

- Generative AI Overview (5 minutes)
 - What is generative AI?
 - Difference between traditional AI (classification, regression) vs. generative models.
 - Examples: AI-powered coding assistants, SQL generation tools, AI document summarization.
 - How LLMs work at a high level
 - Transformer models, tokens, context windows, and their limitations.
 - Importance of input structure (garbage in, garbage out).
 - Where the models live on AWS
 - Amazon Bedrock: Managed service providing foundational models (FM) from AWS and third-party providers (Anthropic, Mistral, etc.).
 - SageMaker: Pre-trained models with AWS integration.
- Introduction to LangChain with AWS (5-7 minutes)
 - Why LangChain?
 - Simplifies LLM integration for building AI tools with AWS services.
 - Works with Amazon Bedrock, AWS-hosted vector databases, and S3 storage.
 - Core LangChain components:
 - LLM wrappers: Connecting LangChain to Amazon Bedrock models.
 - Chains: Building AI workflows for structured responses.
 - Memory: Allowing models to retain short-term context.
 - Running LangChain on AWS:
 - Accessing AWS-hosted LLMs through Amazon Bedrock.
- Hands-on Activities Overview (3 minutes)
 - Introduce the three topic areas (code generation, structured data, unstructured data).
 - Explain that students will choose one or explore multiple as they modify starter code.
 - Walk through the AWS-hosted Jupyter notebook setup with access to Bedrock models.

Hands-on Activities

Activity 1: Getting Started with Bedrock and Basic Prompting

Goal: Get students comfortable with running an AWS-hosted LLM using LangChain and Amazon Bedrock, general LLM prompting.

Starter Code: Pre-written notebook that connects LangChain to Amazon Bedrock, invoking a foundational model hosted on AWS.

Steps:

1. Authenticate with AWS and configure Bedrock access.
2. Load a model from Bedrock.
3. Run a simple prompt and print the output.
4. Modify model parameters (e.g., temperature, max tokens) and observe the changes.
5. Create a chain with prompts and output parsing.

Activity 2: Prompt Engineering

Goal: Understand how LangChain chains work through human and system prompts, prompt engineering through zero-shot, few-shot, and chain-of-thought (CoT) prompting as well as adding chat memory.

Starter Code: Notebook with a basic LangChain chain that takes an input query, processes it through a Bedrock-hosted model, and returns a response.

Steps:

1. Explore zero-shot, few-shot, and CoT prompts.
2. Add memory to the chain.
3. Learn some best practices for prompt engineering.

Activity 3: Structured and Custom Output Formatting

Goal: Learn how to parse the LLM output into useful formats.

Starter Code: Notebook with a LangChain pipeline that takes a query, processes it, and returns a response.

Steps:

1. Add a JSON output parser to the chain.
2. Update the prompt to provide more reliable JSON output for parsing.
3. Create a custom parser and add it to the chain.
4. Explore parsing YAML.

Tasks

Code Generation (Dataset: two different pieces of code)

1. Explain how you would generate boilerplate code for a payment processing microservice in Python that integrates with a payment gateway API. In your explanation, include key considerations for security (e.g., token-based authentication, encryption), error handling, and logging. Provide a sample code snippet.

2. An S3 bucket contains a COBOL code block from a mainframe-based payment processing system. Analyze the provided COBOL program and describe in detail its functionality.
3. You are given a Python code snippet (`my_bad_code.py` in the S3 bucket) that generates API endpoint templates using Flask and Jinja2. The provided code is poorly written: it has no comments, no docstrings, and no type hints. Your task is to refactor this 'bad' code into a 'good' version that adheres to Python best practices.
4. Generate synthetic invoice data for testing an `is_overdue` function. Produce the data directly (do not output Python code) as a JSON array containing exactly 5 invoice objects. Each invoice object must include the following keys: `invoice_id`, `customer_name`, `invoice_date`, `due_date`, `amount`. The final amount should be a JSON-formatted dataset.
5. You are working on a failing function that calculates accrued interest. (Function provided in notebook.) Create a prompt using chain-of-thought prompting that walks through identifying the bug, explaining why it happens, and suggesting a corrected version of the code. The model must reason step-by-step before generating the new code.
6. You are provided a function. (Function provided in the notebook.) When you run it you get the following exception: `Error occurred: [Errno 21] Is a directory: '/tmp/'` Generate a prompt that takes the exception and returns:
 - A layperson-friendly description of what went wrong
 - Suggested fix for the code

Be sure you are not just repeating the traceback.

7. Create a YAML linter to generate a YAML config with three Python linter rules for a codebase. Each rule must include:
 - `rule_name`: short, descriptive string
 - `severity`: "warning" or "error"
 - `description`: brief explanation

The rules you must generate are:

1. `no-hardcoded-credentials` — flag hardcoded secrets like API keys or DB passwords
2. `validate-currency-format` — ensure currency values are consistently formatted
3. `use-typed-annotations` — encourage use of Python type hints for clarity

Use a custom output parser that:

- Strips markdown/code fences
- Parses the YAML with `yaml.safe_load()`
- Validates that rules is a list of 3 dicts, each with the required keys

Your final output must be valid YAML only — no markdown, no extra text.

8. You're simulating the output of a Copilot-style AI assistant that generates markdown-heavy documentation for utility functions.

Your task:

1. Write a prompt that asks the model to generate documentation for a Python function called `reconcile_transactions`. This function compares two sets of financial transaction records (e.g., internal ledger vs. bank statement) and identifies matches, mismatches, and discrepancies.
2. Instruct the model to include extra commentary, formatting, and markdown — but embed a structured JSON object within the text.
3. The embedded JSON object must contain the following keys:
 - `function_name`: string
 - `dependencies`: list of strings
 - `logic_summary`: 1-sentence summary of what the function does
 - `notes`: list of short bullet points describing key features or implementation details
4. Write a custom parser that extracts the JSON object from the noisy output. Your parser should:
 - Use regex to find the first `{ ... }` block
 - Parse it using `json.loads()`
 - Return only the cleaned JSON object

The final output should be a clean Python dictionary representing the embedded JSON block.

Structured Data Analysis (Dataset: e-commerce CSV file)

1. Using the dataset, identify the top three best-selling products in each country based on total quantity sold. For each product, calculate the total quantity sold and the total revenue ($\text{Quantity} \times \text{UnitPrice}$). Return your answer as a JSON object where each key is a country and its value is a list of the top three products with their 'Description', 'Total Quantity Sold', and 'Total Revenue'.
2. Analyze the dataset to compute the average spending per invoice for each country. For every invoice, sum the product of Quantity and UnitPrice to determine its total, then average these totals for each country. Identify which country has the highest average invoice spending. Return your answer in JSON format with each country and its average spending, along with a separate field for the country with the highest average.
3. Examine the 'InvoiceDate' field to determine if there are any seasonal patterns in the sales data. Identify the time period (day, week, or month) that shows the highest total sales revenue (calculated as $\text{Quantity} \times \text{UnitPrice}$). Provide supporting metrics such as total revenue for each period and any observed trends. Return your analysis in JSON format with clear supporting metrics for each period.
4. Determine the total sales revenue for each day of the week (Monday, Tuesday, etc.) from the provided e-commerce dataset. For each transaction, compute revenue as Quantity multiplied by UnitPrice, and use the InvoiceDate to determine the day of the week. Then, aggregate the revenue by day of the week and identify which day has the highest total revenue. Return your answer as a JSON object where each key is a day of the week

mapped to its total revenue, and include an extra key 'day_with_highest_revenue' indicating the day with the maximum total revenue.

5. Using the file `balance_sheet.csv` in the S3 bucket, provide a JSON array output of the keys `Year`, `TotalAssets`, `CashAndSTInvestments`, and `CashSTGrowth`, populating these values for each year from the data.
6. Using the file `balance_sheet.csv`, compute the liquidity ratio for each year (defined as `Cash & Short Term Investments` divided by `Total Assets`) and calculate the year-over-year asset growth percentage for each consecutive pair of years. Then, output your results as a valid JSON object with two keys: `"LiquidityMetrics"` (an array of objects, each containing the year and its corresponding liquidity ratio) and `"AssetTrends"` (an array of objects for each year, starting from the second, with the computed asset growth percentage).
7. Given the earnings report available in `full_text.txt` in the S3 bucket, extract the condensed consolidated balance sheets and output that information in a markdown table.
8. Using the file `full_text.txt`, extract year-over-year financial data for three metrics—gross profit, total operating expenses, and net income and calculate the differences and percentage changes. The output must be a JSON array where each element is an object containing the keys `"metric"`, `"current"`, `"previous"`, `"difference"`, and `"percentage_change"`. Each value should be represented as a string with appropriate units (e.g., `"100.0 million"` or `"10.0%"`), and no additional text or separate JSON arrays should be included.

Unstructured Data Analysis (Dataset: text from 2025Q2 press release)

1. Calculate the previous year's Q2 figures for subscription fees and transaction fees given that the current quarter's subscription fees are \$67.7 million (up 7% YoY) and transaction fees are \$251.9 million (up 19% YoY). Then, verify that the sum of these base amounts aligns with the reported overall 16% increase to a current core revenue of \$319.6 million. Show all your work and calculations.
2. Analyze the Q2 FY25 actual financial results presented in the document. Discuss how the changes in core revenue, total revenue, operating loss, and net income reflect the company's performance, and explain what these figures might indicate about the company's overall growth trajectory and operational efficiency. Provide a detailed analysis with supporting insights.
3. Summarize the key differences between GAAP and non-GAAP financial measures as detailed in the press release. Identify which items are excluded from the non-GAAP calculations and discuss the rationale behind these exclusions from the perspective of financial analysis and investor decision-making.
4. From the forward-looking statements section, identify at least three major risks or uncertainties that BILL highlights as potentially affecting future performance. Provide a detailed analysis of how each risk could impact the company's financial results and the overall business outlook.

5. Given the file full_text.txt in the provided S3 bucket, create a JSON output that has the keys Goodwill, Stockholders_Equity_Common_Stock and Total_Revenue. The values should be a list of all values found for these keys.
6. Utilizing the file full_text.txt, you have been asked to explore trends in the stock-based compensation expenses. Create an email to Gaurav Tandon with this analysis.
7. Utilizing the file schedule_14a.txt in the S3 bucket, create a list of JSON based on this file where the keys are Proposal name, Proposal text, and Board recommendation. Find all proposals under vote in this document and populate the values with the relevant information.
8. Using the file schedule_14a.txt, create a list of JSON with the keys name, role, and term expiration. Populate it with the members of the board.

Wrap-up

- Quick discussion: What worked? What didn't?
- Address common issues.
- Preview of Module 2: Next, students will learn to integrate external data sources and retrieval-augmented generation (RAG) to further improve model performance through the mitigation of hallucinations

Summary of Learning Outcomes

- Authenticate and use AWS-hosted models in Amazon Bedrock with LangChain.
- Run and modify LLM pipelines on AWS.
- Understand and customize LangChain chains for different use cases.
- Connect LangChain with AWS services.
- Apply foundational AI concepts to code generation, structured data, and unstructured data applications.
- Understand the impact of different prompting strategies on model outputs
- Modify prompts to improve response quality and clarity
- Tune model parameters to control randomness, response length, and specificity
- Structure model outputs in JSON, YAML, or other machine-readable formats
- Apply prompt engineering techniques to code generation, structured data analysis, and unstructured data processing

Module 2: Retrieval-Augmented Generation (RAG) & External Data Integration

Objective: Teach students how to enhance LLM responses by integrating external data sources using retrieval-augmented generation (RAG). This module will cover embeddings, vector search, and structured retrieval techniques using AWS services.

Lecture

- Introduction to Retrieval-Augmented Generation (5 minutes)
 - What is RAG, and why is it important?
 - LLMs do not have up-to-date or domain-specific knowledge
 - Reduces hallucinations by grounding responses in retrieved data
 - Two-step process:
 - Retrieve relevant context from an external source
 - Use the retrieved context to generate a response
- Data Sources and AWS Services for Retrieval (5-7 minutes)
 - Types of data sources:
 - Structured (databases, CSVs, APIs)
 - Unstructured (documents, PDFs, web pages)
 - AWS services for implementing RAG:
 - Amazon OpenSearch: Vector database for storing and retrieving text embeddings
 - Amazon RDS/DynamoDB: Querying structured data for context
 - Amazon S3: Storing large unstructured documents for retrieval
 - Amazon Bedrock: Hosting and running LLM inference with integrated retrieval
 - Overview of embeddings and vector search
 - How text is converted into numerical representations for retrieval
 - Storing and querying embeddings efficiently
- Hands-on Activities Overview (3 minutes)
 - Students will implement a retrieval system using AWS-hosted services
 - Activities will include indexing and retrieving structured and unstructured data
 - AWS-hosted Jupyter notebooks will be provided with starter code

Hands-on Activities

Activity 1: Creating an Embeddings Index

Goal: Store and retrieve text embeddings using a vector database.

Starter Code: Notebook with a pre-built embedding model from Amazon Bedrock.

Steps:

1. Convert sample text data into embeddings using an Amazon Bedrock-compatible model.

2. Store embeddings in a vector database.
3. Implement a simple query to retrieve relevant embeddings.

Activity 2: Implementing a RAG Pipeline

Goal: Enhance LLM responses by retrieving relevant external data before generation.

Starter Code: Notebook with a simple RAG implementation using Amazon OpenSearch.

Steps:

1. Retrieve relevant text snippets from the embedding index.
2. Pass retrieved text as additional context to the LLM.
3. Compare results with and without retrieval.

Activity 3: Integrating an AWS Data Source

Goal: Connect an AWS data source (S3, RDS, DynamoDB) to the retrieval pipeline.

Starter Code: Notebook with API calls to AWS services.

Steps:

1. Load a dataset from Amazon S3, RDS, DynamoDB, etc.
2. Pre-process and embed the data for retrieval.
3. Integrate retrieval into the RAG workflow.

Tasks

Code Generation

For the questions in this module you will be using the file `api_docs.txt` in the S3 bucket, which is a text scraping of the Bill v3 API docs. You will need to create a RAG out of these docs for answering the following questions.

1. Write a function that will create a vendor in the API Sandbox with the following information (provided in the notebook).
2. Write a function that will create an invoice for a random amount between 100.00 - 1000.00 dollars when provided with a vendor ID.
3. Write a function that will return all unpaid invoices from the API into the following format (provided in the notebook).
4. Write a function that will return the total outstanding amount due for each vendor in an organization.

Structured Data Analysis

For these tasks you will be working with the file 2025Q2_data_cleaned.xlsx, which is in the S3 bucket. Before you begin these tasks, you are strongly encouraged to look at that file to get a feel for what is in there and develop strategies for how you will ingest this data!

In the answering of the following questions, be sure to consider everything we have talked to until this point, including both model and prompt optimization. You also might find it useful to add memory to your chain so that you can continue to ask questions until you land on the answer you like for your submissions.

1. Utilizing the provided data, determine the percent change in subscription fees between 2023 and 2024.
2. For the three-month period in 2024, calculate the operating margin and explain in a few sentences what this margin indicates about the company's performance.
3. Summarize the key changes in revenue, costs, and profits between 2023 and 2024.
4. Between the 2023 and 2024 data what are the revenue trends and their drivers?

Unstructured Data Analysis

For the tasks for this module you will need to create a RAG utilizing the following files in the S3 bucket:

- Bill-q2-2025-10-q.pdf
- BILL-Q2-25-Investor-Deck.pdf
- BILL-Q2-25-Press-Release-2-6-25.pdf
- Bill-10-k-6-30-2024.pdf

You may also bring in any additional data sources you choose (although you should consider the following questions and use them to guide what additional data might be helpful if you choose to do so).

In the answering of the following questions, be sure to consider everything we have talked to until this point, including both model and prompt optimization. You might find it helpful to go beyond the `RetrievalQA.from_chain_type()` in Module 3 Activity 3 and explore using `create_stuff_documents_chain` and `create_retrieval_chain`. You also might find it useful to add memory to your chain so that you can continue to ask questions until you land on the answer you like for your submissions.

1. Describe BILL's financial performance and strategic outlook as of December 31, 2024. Be sure to indicate the source(s) of the information used to create this perspective.
2. What primary drivers for future performance are consistently emphasized by BILL?

3. Considering the strategic initiatives detailed in the investor deck and press release, how does management justify these plans against the backdrop of market challenges and potential financial headwinds identified in the 10-Q statement?
4. Based on the provided files and any additional data you used, what might be a new product that you could recommend that BILL develop to enhance its existing platform?

Wrap-up

- Quick discussion: What worked? What didn't?
- Address common issues.
- Discuss how retrieval improved response accuracy
- Review of key takeaways from RAG implementation
- Preview of Module 4: Next, students will learn about LangChain agents, tool integration, and model evaluation

Summary of Learning Outcomes

- Understand the role of RAG in improving LLM accuracy
 - Create and query embeddings using Amazon OpenSearch and Amazon Bedrock
 - Implement a retrieval pipeline to provide external context to an LLM
 - Integrate AWS data sources (S3, RDS, DynamoDB) for structured and unstructured retrieval
 - Apply retrieval techniques to code generation, structured data, and unstructured data processing
-

Module 3: Agents & Tool Use

Objective: Teach students how to build AI-powered agents that interact with external tools and APIs, while also evaluating their model's effectiveness. By the end of this module, students will be able to create multi-step workflows using LangChain agents, integrate external APIs, and assess model performance using quantitative and qualitative evaluation methods.

Lecture

- Introduction to AI Agents (5 minutes)
 - What is an AI agent?

- Unlike standard LLM pipelines, agents make decisions dynamically based on user input.
 - Can use external tools (APIs, databases, search engines) to gather additional information.
- When to use agents:
 - Automating workflows that require decision-making.
 - Enhancing model reliability by retrieving up-to-date information.
 - Building AI-powered assistants that interact with APIs and databases.
- Types of agents in LangChain:
 - ReAct Agents: Combine reasoning and action.
 - OpenAI Function-Calling Agents: Execute predefined tool calls.
 - Self-Ask Agents: Break down complex queries into multiple steps.
- Connecting Agents to External Tools (5-7 minutes)
 - Overview of tool integration:
 - APIs (e.g., AWS services, third-party APIs).
 - Database queries (DynamoDB, RDS).
 - File storage (retrieving documents from S3).
 - Implementing function-calling capabilities:
 - Structuring API responses for seamless LLM interaction.
 - Chaining multiple tool calls for multi-step automation.
 - Example use cases:
 - AI-powered SQL assistants that generate, validate, and execute queries.
 - Coding assistants that retrieve documentation before generating code.
 - AI document processors that extract and analyze structured information.
 - Model Evaluation Using LLM-Based Techniques (3 minutes)
- Why evaluating AI performance is critical.
 - Basic evaluation approaches:
 - Self-Consistency Testing: Ask the model the same question multiple times and compare responses.
 - LLM-Based Scoring: Use another LLM to assess quality, relevance, and correctness.
 - Comparison to Reference Outputs: Compare AI-generated results against a known good answer.
 - Importance of human-in-the-loop feedback.

Hands-on Activities

Activity 1: Building a Simple AI Agent

Goal: Create an agent that decides when to use external tools based on user input.

Starter Code: Notebook with a basic LangChain agent running on Amazon Bedrock.

Steps:

1. Implement a basic ReAct agent that reasons before responding.
2. Define a set of tools (e.g., API calls, SQL query execution, document retrieval, etc.).
3. Modify the agent to choose the correct tool based on user input.

Activity 2: Integrating Other Services into an Agent

Goal: Extend the agent's functionality by enabling it to call an external API or AWS service.

Starter Code: Notebook with predefined API endpoints and AWS service integrations.

Steps:

1. Connect the agent to an external API (e.g., OpenSearch for vector search, S3 for document retrieval, RDS for database queries).
2. Implement function calling to fetch real-time data when needed.
3. Modify the workflow to pass retrieved data as context to the LLM.

Activity 3: Evaluating and Refining Model Outputs Using LLM-Based Techniques

Goal: Assess model performance using self-consistency testing, LLM-based evaluation, and reference comparisons.

Starter Code: Notebook with evaluation functions that compare AI responses.

Steps:

1. Implement a self-consistency check by asking the model the same question multiple times and comparing outputs.
2. Use an LLM as a grader to score AI-generated responses based on relevance, accuracy, and structure.
3. Compare AI-generated outputs against reference answers and measure differences.

Tasks

All topic areas will have common tasks for this module.

In this final set of challenges we will no longer be working in tracks, but rather on a unified set of problems. You will be utilizing all of the skills you have learned throughout the course of the workshop.

You will be provided with access to a BILL Sandbox that has data preloaded into it. Using the v3 API, your task will be to create an agent that accesses the API and any other things you might choose. Using this agent, you will need to complete the following.

1. Within the Sandbox you will find a vendor named "San Jose Music Supplies." You will need to determine how much money they owe this organization as well as how late their required payments are. Using this information, construct an email to them reminding them of their outstanding balance and requesting payment. Use language that is appropriate to how late their payments are.
2. Also within the Sandbox is the vendor named "Acme Auto Parts" that has some outstanding payments. Construct an email to them reminding them of their outstanding balance and requesting payment. Use language that is appropriate to how late their payments are.
3. You are the accountant for this simulated organization and have to provide a summary of the outstanding balances due to the organization. Obtain this information and use it to craft an email to your boss with this information.
4. Based on the data in the Sandbox, forecast potential cash flow issues based on increasing overdue trends and recommend actions such as changes to payment terms or prioritization of collections.

Wrap-up

- Quick discussion: What worked? What didn't?
- Address common issues.
- Final Q&A and next steps for applying LangChain in production.

Summary of Learning Outcomes

- Understand the role of AI agents and their decision-making processes.
 - Integrate external APIs, databases, and AWS services into LangChain workflows.
 - Implement basic LLM-based evaluation techniques to measure model accuracy and consistency.
 - Debug and optimize LLM responses for different task areas.
 - Apply agent-based workflows to code generation, structured data, and unstructured data applications.
-

Final Presentations and Wrap-Up

- Team presentations (OPTIONAL, 5-7 minutes each)
- Final scores presented
- Closing remarks and key takeaways